

1 - Introduction to Python

April 27, 2026

Giới thiệu Python

TS. Lê Thành Trung

April 27, 2026

Contents

1	Tại sao nên dùng Python?	3
2	Môi trường lập trình	3
2.1	Cài đặt Python và VSCode	3
2.2	Giới thiệu Jupyter Notebook (.ipynb)	3
2.2.1	Cài đặt Extensions Jupyter trên VSCode	3
2.2.2	Cách tạo hoặc mở file .ipynb	3
2.2.3	Điểm mạnh	4
2.3	Giới thiệu Google Colab	4
2.3.1	Google Colab là gì?	4
2.3.2	Điểm mạnh	4
2.3.3	Hạn chế	5
3	Giới thiệu Mô-đun (Module), Gói (Package) và Thư viện (Library)	6
3.1	Mô-đun (Module) – Đơn vị cơ bản	6
3.2	Gói (Package) - Cấu trúc thư mục	6
3.2.1	Quản lý Gói bằng Terminal	6
3.2.2	Quản lý Gói ngay trong Jupyter Notebook	6
3.3	Thư viện (Library) – Tập hợp chức năng	7
4	PEP 8: Nghệ thuật viết code sạch và dễ đọc	7
4.1	Giới thiệu PEP 8	7
4.2	Tóm tắt một số quy tắc cơ bản	8
4.2.1	Cách trình bày văn bản (Layout)	8
4.2.2	Khoảng trắng trong biểu thức (Whitespace)	8
4.2.3	Quy ước đặt tên (Naming Conventions)	8
4.2.4	Chú thích (Comments)	8
4.2.5	Thứ tự Import thư viện	9
5	Chương trình đầu tiên	9
5.1	Trong Jupyter Notebook	9
5.2	Trong file .py	9

1 Tại sao nên dùng Python?

- **Phổ biến nhất hiện nay:** Python liên tục dẫn đầu bảng xếp hạng ngôn ngữ lập trình (nguồn [TIOBE Index](#)), đồng nghĩa với cộng đồng hỗ trợ lớn, tài liệu phong phú và dễ tìm kiếm giải pháp.
- **Đơn giản, dễ học:** Cú pháp gần với ngôn ngữ tự nhiên, cho phép bạn tập trung vào giải pháp chứ không phải cú pháp. Người mới bắt đầu có thể viết chương trình chạy được chỉ sau vài giờ.
- **Năng suất cao:** Viết ít dòng code hơn đáng kể so với Java hay C++, giúp rút ngắn thời gian phát triển và dễ bảo trì.
- **Đa năng:** Dùng được trong nhiều lĩnh vực — khoa học dữ liệu, trí tuệ nhân tạo/học máy, thống kê, phân tích tài chính, tự động hoá, phát triển web, lập trình khoa học (scientific computing).
- **Kho thư viện khổng lồ:** Hệ sinh thái gói rất phong phú (NumPy, pandas, scikit-learn, TensorFlow, matplotlib...), liên tục được cộng đồng phát triển và cập nhật.
- **Tích hợp tốt với các công cụ khác:** Kết nối dễ dàng với SQL, Excel, R, Spark, các API và dịch vụ đám mây — rất phù hợp cho người làm phân tích dữ liệu và thống kê.
- **Miễn phí và mã nguồn mở:** Python ra mắt phiên bản đầu tiên vào năm 1991 và hiện tại là phiên bản **3.13**. Không mất chi phí bản quyền, có thể sử dụng tự do cho cả mục đích cá nhân lẫn thương mại.
- **Khả năng di chuyển (portable):** Chạy được trên Windows, macOS và Linux mà hầu như không cần thay đổi mã nguồn.
- **Khả năng mở rộng:** Có thể nhúng code C/C++ hoặc Fortran vào Python để tối ưu hiệu năng khi cần thiết.
- **Cơ hội nghề nghiệp:** Nhu cầu tuyển dụng Python developer và data scientist liên tục tăng, mức lương cạnh tranh trên thị trường toàn cầu.

2 Môi trường lập trình

2.1 Cài đặt Python và VSCode

- Xem hướng dẫn trong file đính kèm!

2.2 Giới thiệu Jupyter Notebook (.ipynb)

2.2.1 Cài đặt Extensions Jupyter trên VSCode

Gồm các bước sau:

1. Mở VSCode, nhấn vào biểu tượng Extensions ở thanh bên trái (hoặc bấm tổ chức phím Ctrl + Shift + X).
2. Tìm kiếm từ khóa “Jupyter”.
3. Nhấn Install gói extension chính tên là Jupyter (của Microsoft). Việc này sẽ tự động cài đặt thêm các tiện ích liên quan như Jupyter Keymap và Jupyter Cell Tags.

2.2.2 Cách tạo hoặc mở file .ipynb

- Cách tạo file: File -> New File -> Jupyter Notebook

- Cách mở file `.ipynb` có sẵn: chỉ cần click chuột vào file `.ipynb` trong trình quản lý file của VSCode, giao diện Notebook sẽ hiện ra ngay lập tức.

2.2.3 Điểm mạnh

- **Sự kết hợp “3 trong 1” (Văn bản - Mã nguồn - Kết quả):** lợi ích lớn nhất của Notebook là khả năng tạo ra một tài liệu sống. Thay vì chỉ có các dòng code khô khan, một file `.ipynb` bao gồm:
 - Văn bản định dạng (Markdown): Giúp giải thích lý thuyết, chèn hình ảnh, bảng biểu và công thức toán học.
 - Mã nguồn (Code): Các đoạn chương trình Python có thể chạy được.
 - Kết quả (Output): Hiển thị ngay lập tức các con số, bảng dữ liệu hoặc đồ thị ngay dưới đoạn code vừa chạy.
- **Tư duy lập trình theo từng khối (Cell-based Execution):** khác với việc chạy một file script `.py` dài từ đầu đến cuối, Notebook chia chương trình thành các ô (cells) độc lập:
 - Chạy từng phần: Người học có thể chạy thử một đoạn code nhỏ, kiểm tra kết quả, sau đó mới viết tiếp đoạn sau.
 - Lưu trữ trạng thái: Khi một ô đã chạy, các biến dữ liệu sẽ được lưu vào bộ nhớ. Bạn có thể thay đổi code ở ô dưới và chạy lại nhiều lần mà không cần phải thực thi lại các bước tính toán nặng nề ở các ô phía trên.
- **Tiêu chuẩn công nghiệp hiện nay:** việc làm quen với Jupyter Notebook giúp sinh viên tiếp cận sớm với công cụ tiêu chuẩn mà các kỹ sư dữ liệu, nhà khoa học và lập trình viên Python chuyên nghiệp trên thế giới đang sử dụng hàng ngày.

2.3 Giới thiệu Google Colab

2.3.1 Google Colab là gì?

Google Colaboratory (gọi tắt là Colab) là một phiên bản Jupyter Notebook chạy hoàn toàn trên nền tảng đám mây của Google. Thay vì chạy Python trên máy tính cá nhân, bạn chạy nó trên các máy chủ mạnh mẽ của Google thông qua trình duyệt web. Truy cập <https://colab.research.google.com/>.

2.3.2 Điểm mạnh

- **Không cần cài đặt (Zero Configuration):** đây là lợi ích lớn nhất cho người học. Bạn không cần cài đặt Python, Anaconda hay bất kỳ thư viện nào (như NumPy, Pandas, Matplotlib) lên máy tính. Chỉ cần có một tài khoản Google và trình duyệt web là có thể bắt đầu lập trình ngay lập tức. Điều này giúp tiết kiệm 1-2 buổi học đầu tiên chỉ để loay hoay sửa lỗi cài đặt.
- **Miễn phí tài nguyên phần cứng mạnh mẽ (GPU/TPU):** Google cung cấp miễn phí quyền truy cập vào các bộ xử lý đồ họa (GPU) và bộ xử lý chuyên dụng (TPU). Lợi ích: Đối với các bài toán tính toán nặng hoặc học máy (Machine Learning), máy tính cá nhân của người học thường không đủ mạnh, nhưng Colab có thể xử lý dễ dàng.
- **Khả năng chia sẻ và cộng tác:** Bạn có thể chia sẻ file Notebook cho đồng nghiệp chỉ bằng một đường link. Giảng viên và sinh viên có thể cùng làm việc trên một file, viết nhận xét (comment) và sửa lỗi trực tiếp cho nhau theo thời gian thực.

- **Lưu trữ an toàn trên Google Drive:** Mọi thay đổi trong code được tự động lưu vào Google Drive. Bạn không bao giờ lo mất bài làm nếu máy tính hỏng hoặc quên lưu file. Dễ dàng quản lý bài tập theo từng thư mục trên Drive.

2.3.3 Hạn chế

- **Phụ thuộc hoàn toàn vào Internet:** Vì chạy trên nền tảng đám mây, nếu mất kết nối mạng, bạn không thể viết code, chạy code hay truy cập vào dữ liệu.
- **Thời gian sống của phiên làm việc (Runtime Timeout):** Google Colab có cơ chế tự động ngắt kết nối (Disconnect) nếu bạn không tương tác trong một khoảng thời gian (thường là 30-90 phút) hoặc khi phiên làm việc kéo dài quá 12 tiếng.
- **Giới hạn tài nguyên miễn phí:** Mặc dù cung cấp GPU miễn phí, nhưng không phải lúc nào cũng có sẵn. Google ưu tiên tài nguyên cho người dùng trả phí (Colab Pro).
- **Khó khăn khi làm việc với các file cục bộ:** Không giống như Jupyter chạy trên máy cá nhân (có thể đọc file ngay trong thư mục ổ cứng), Colab cần các bước trung gian để đọc dữ liệu (phải upload lên, dùng lệnh để mount Google Drive, hoặc dùng API để chọn file).
- **Thiếu tính năng “IntelliSense” mạnh mẽ:**
 - VSCode: Cung cấp khả năng gợi ý code (autocompletion) cực nhanh, giải thích chi tiết các tham số của hàm, và kiểm tra lỗi cú pháp ngay khi bạn vừa gõ xong (real-time).
 - Colab: Gợi ý code chậm hơn (vì phải gửi yêu cầu lên server) và đôi khi không hiển thị đầy đủ các tài liệu hướng dẫn (docstrings) của hàm như VSCode.
- **Trình quản lý biến (Variable Explorer) hạn chế:**
 - VSCode: Có một bảng điều khiển riêng biệt, hiển thị cực đẹp và trực quan danh sách tất cả các biến, kích thước mảng (shape), kiểu dữ liệu. Bạn có thể nhấn vào một mảng NumPy để xem nó dưới dạng bảng như Excel.
 - Colab: Cũng có bảng quản lý biến nhưng giao diện đơn giản hơn, khó quan sát và thao tác hơn khi số lượng biến lớn.
- **Khả năng tùy chỉnh giao diện (Customization):**
 - VSCode: Bạn có thể cài hàng nghìn Theme (màu sắc), thay đổi font chữ, biểu tượng, sắp xếp các cửa sổ (chia đôi màn hình để vừa xem code vừa xem đồ thị) theo ý thích cá nhân.
 - Colab: Giao diện rất cố định, chỉ có chế độ Sáng/Tối và một vài tùy chỉnh nhỏ. Việc chia màn hình để làm việc đa nhiệm trên Colab rất khó khăn.
- **Thiếu các công cụ hỗ trợ lập trình chuyên nghiệp:**
 - Debugger (Trình gỡ lỗi): VSCode có bộ Debugger cực mạnh, cho phép bạn đặt “Breakpoints” để dừng chương trình tại một dòng cụ thể và soi xét từng ngóc ngách của bộ nhớ. Colab hỗ trợ Debug rất thô sơ và khó dùng hơn.
 - Cấu trúc thư mục: Trong VSCode, bạn nhìn thấy toàn bộ cấu trúc dự án (nhiều file .py, .ipynb, thư mục ảnh, dữ liệu) ở thanh bên trái rất chuyên nghiệp. Colab quản lý file theo dạng danh sách của máy ảo, nhìn rất rối mắt và khó thao tác với các dự án có nhiều tệp tin liên kết.
- **Phím tắt và Thao tác chuột**
 - VSCode: Hỗ trợ đa con trỏ (Multi-cursor) giúp sửa nhiều dòng code cùng lúc, phím tắt linh hoạt và mượt mà.
 - Colab: Phụ thuộc vào trình duyệt nên đôi khi phím tắt bị xung đột với phím tắt của

trình duyệt (như Ctrl+W, Ctrl+T), thao tác phản hồi luôn có độ trễ nhất định.

3 Giới thiệu Mô-đun (Module), Gói (Package) và Thư viện (Library)

3.1 Mô-đun (Module) – Đơn vị cơ bản

- Định nghĩa: Một Module đơn giản là một tệp tin có đuôi `.py`.
- Bản chất: Nó chứa các định nghĩa hàm, lớp (class), hoặc biến. Mục đích của module là chia nhỏ mã nguồn để dễ quản lý và tái sử dụng.
- Ví dụ: Bạn tạo một file `thacsithongke.py`. Dùng `import thacsithongke` để sử dụng các hàm, lớp, hoặc biến trong đó.

3.2 Gói (Package) - Cấu trúc thư mục

- Định nghĩa: Package là một tập hợp các Module được tổ chức trong một thư mục.
- Đặc điểm nhận dạng: Để Python hiểu một thư mục là một Package, bên trong thư mục đó thường chứa một tệp đặc biệt tên là `__init__.py` (từ phiên bản Python 3.3 trở đi thì không bắt buộc, nhưng vẫn nên có).

3.2.1 Quản lý Gói bằng Terminal

- `pip` là trình quản lý gói mặc định của Python, cho phép tải, cài đặt, cập nhật và gỡ bỏ các gói từ kho [PyPI](#) chỉ bằng một dòng lệnh trong Terminal.
- **Kiểm tra**
 - `pip --version`
- **Cài đặt**
 - `pip install <tên-gói>`
 - `pip install <tên-gói>==<phiên-bản>`
- **Cập nhật & gỡ**
 - `pip install --upgrade <tên-gói>`
 - `pip uninstall <tên-gói>`
- **Thông tin**
 - `pip list`
 - `pip show <tên-gói>`
- **Xuất danh sách**
 - `pip freeze > requirements.txt`

3.2.2 Quản lý Gói ngay trong Jupyter Notebook

Trong Jupyter Notebook (`.ipynb`), bạn có thể chạy lệnh hệ thống trực tiếp trong một **code cell** bằng cách thêm dấu `%` ở đầu lệnh để đảm bảo gói được cài đúng vào môi trường Python mà kernel đang dùng.

- Cài đặt gói

```
[ ]: # Cách khuyến nghị trong notebook
      %pip install numpy
```

- Cập nhật gói

```
[ ]: # Cập nhật pip
%pip install --upgrade pip
```

```
[ ]: # Cập nhật gói lên phiên bản mới nhất
%pip install --upgrade numpy
```

- Gỡ bỏ gói

```
[ ]: # Gỡ bỏ gói (tự động xác nhận với -y)
%pip uninstall numpy -y
```

- Các lệnh hữu ích khác

```
[ ]: # Liệt kê tất cả các gói đã cài
%pip list
```

```
[ ]: # Xem thông tin chi tiết một gói
%pip show pandas
```

```
[ ]: # Xuất danh sách gói ra file requirements.txt
%pip freeze
```

3.3 Thư viện (Library) – Tập hợp chức năng

- Định nghĩa: Library là một thuật ngữ rộng hơn, không có cấu trúc file cụ thể trong Python. Nó mô tả một tập hợp các Package và Module được thiết kế để giải quyết một nhóm vấn đề cụ thể.
- Bản chất: Một Library có thể bao gồm hàng chục Package con liên kết với nhau.
- Ví dụ thực tế: SciPy là một Library khổng lồ. Bên trong nó có các Package như `scipy.integrate`, `scipy.optimize`, `scipy.fft`.
- Cài đặt: giống như package.

4 PEP 8: Nghệ thuật viết code sạch và dễ đọc

4.1 Giới thiệu PEP 8

- “Code được đọc nhiều hơn là được viết.” — Guido van Rossum (Cha đẻ ngôn ngữ Python).
- **Định nghĩa:** PEP 8 (Python Enhancement Proposal #8) là một bản hướng dẫn cung cấp các quy tắc và quy ước về cách viết mã Python sao cho tối ưu nhất. Nó không bắt buộc để chương trình có thể chạy, nhưng nó là “luật chơi chung” của cộng đồng lập trình viên Python trên toàn thế giới.
- **Lợi ích cốt lõi:**
 - Tính nhất quán (Consistency): Giúp mã nguồn của bạn trông giống như được viết bởi một chuyên gia, dù bạn làm việc một mình hay trong nhóm.
 - Dễ đọc (Readability): Giúp đồng nghiệp, giảng viên (và chính bạn trong tương lai) dễ dàng hiểu được logic của chương trình mà không tốn quá nhiều công sức.

- Dễ bảo trì: Code sạch giúp việc tìm lỗi (debug) và nâng cấp sau này trở nên nhanh chóng hơn.
- Việc tuân thủ PEP 8 không chỉ giúp bạn trở thành một lập trình viên giỏi, mà còn thể hiện sự tôn trọng đối với những người sẽ đọc code của bạn.
- Tài liệu gốc: [PEP 8 – Style Guide for Python Code](#).

4.2 Tóm tắt một số quy tắc cơ bản

4.2.1 Cách trình bày văn bản (Layout)

- **Thụt lề (Indentation):** Luôn sử dụng 4 khoảng trắng (spaces) cho mỗi cấp độ thụt lề. Không dùng phím Tab (hoặc thiết lập để phím Tab tự chuyển thành 4 khoảng trắng).
- **Độ dài dòng tối đa:** Một dòng code không nên vượt quá 79 ký tự. Điều này giúp có thể mở hai cửa sổ code cạnh nhau mà vẫn đọc được hết nội dung.
- **Dòng trống (Blank Lines):**
 - Cách 2 dòng trống giữa các định nghĩa hàm hoặc lớp (class) ở cấp cao nhất.
 - Cách 1 dòng trống giữa các phương thức bên trong một lớp.

4.2.2 Khoảng trắng trong biểu thức (Whitespace)

- **Xung quanh toán tử:** Luôn có 1 khoảng trắng trước và sau các toán tử gán (=), so sánh (==, <, >, !=), và số học (+, -, *, /).
 - Nên: `x = y + 1`
 - Không nên: `x=y+1`
- **Sau dấu phẩy:** Luôn có 1 khoảng trắng sau dấu phẩy trong danh sách hoặc tham số hàm.
 - Nên: `print(a, b, c)`
 - Không nên: `print(a,b,c)`
- **Tránh khoảng trắng thừa:** Không đặt khoảng trắng ngay sát bên trong ngoặc đơn, ngoặc vuông hay ngoặc nhọn.
 - Nên: `my_list[1]`
 - Không nên: `my_list[ 1 ]`

4.2.3 Quy ước đặt tên (Naming Conventions)

- **Biến và Hàm:** Sử dụng snake_case (chữ thường, cách nhau bằng dấu gạch dưới). Hàm thường bắt đầu với động từ. Ví dụ: `tong_diem`, `tinh_toan_ket_qua()`.
- **Lớp (Class):** Sử dụng PascalCase (Viết hoa chữ cái đầu mỗi từ, không gạch dưới). Ví dụ: `HinhTron`, `SinhVienNghienCuu`.
- **Hằng số:** Sử dụng UPPER_CASE (Viết hoa toàn bộ). Ví dụ: `PI = 3.14`, `MAX_ITERATIONS = 1000`.

4.2.4 Chú thích (Comments)

- **Chú thích khối:** Phải viết thành câu hoàn chỉnh, thụt lề cùng mức với code mà nó giải thích.
- **Chú thích cùng dòng (Inline comments):** Hạn chế dùng. Nếu dùng, phải cách code ít nhất 2 khoảng trắng và bắt đầu bằng # kèm 1 khoảng trắng. Ví dụ: `x = x + 1 # Tang bien dem`

4.2.5 Thứ tự Import thư viện

Các lệnh import luôn đặt ở đầu file, chia thành 3 nhóm theo thứ tự (cách nhau bởi 1 dòng trống):

1. Thư viện chuẩn của Python (ví dụ: os, sys, math).
2. Thư viện bên thứ ba (ví dụ: numpy, pandas).
3. Thư viện tự viết trong dự án.

5 Chương trình đầu tiên

5.1 Trong Jupyter Notebook

```
[1]: print("Hello, World!")
```

Hello, World!

5.2 Trong file .py

- Tạo file .py: File -> New File -> Python File
- Gõ nội dung `print("Hello, World!")` và lưu file với tên `hello.py`
- Thực thi trong VSCode: mở **TERMINAL** ở thư mục có file `hello.py` -> chạy lệnh `python hello.py`.